ESEMPI DI RISPOSTE ESAME

NB: Questi esempi non vanno presi alla lettera, ma dovrebbero darvi una idea del livello di dettaglio richiesto (quindi se alcune cose le scrivete diversamente ma sono CORRETTE, la risposta va bene lo stesso).

In riferimento al file **domande.docx**:

1) Cosa si intende per "spazio metrico" e in che modo la "metrica euclidea su R" vi si relaziona?

Per **SPAZIO METRICO** si intende la coppia (X, d) dove X è un **INSIEME** e d una **DISTANZA**, ovvero:

- $\forall x, y \in X, \quad d(x, y) \geq 0$ **POSITIVITÀ**
- $\forall x, y \in X, \quad d(x, y) = d(y, x)$ **SIMMETRIA**
- $\forall x, y, z \in X, \quad d(x, y) + d(y, z) \geq d(x, z)$ **DIS. TRIANGOL.**

La metrica **EUCLIDEA** è def. su $\mathbb{R}$ da:

$$d(x, y) = \sqrt{(x - y)^2} = |x - y|$$

2) Come si definisce la "continuità" per una funzione f : R^N → R, e qual è il significato di "continuità separata"?

Sia $f : \mathbb{R}^N \to \mathbb{R}$, si dice **CONTINUA** se
$\forall \underline{x} \in \mathbb{R}^N \quad \forall \epsilon > 0 \quad \exists \delta > 0 : \forall \underline{y} \in \mathbb{R}^N$,

$$||\underline{x} - \underline{y}|| \leq \delta \Rightarrow |f(\underline{x}) - f(\underline{y})| < \epsilon$$

Questa definizione implica la continuità in ogni direzione. Se $N = 2$, ovvero se il dominio è in $\mathbb{R}^2$, si può parlare di **CONTINUITÀ SEPARATA** in x e in y.
Ciò significa che la definizione vale solo nella direzione individuata dalla **VARIABILE** x, ovvero $e_1 = (1, 0)$, oppure dalla y, ovvero $e_2 = (0, 1)$, rispettivamente.

3) Qual è la relazione tra "derivata direzionale", "derivata parziale" e "gradiente" nel calcolo in più variabili?

Sia $f : \mathbb{R}^N \to \mathbb{R}$, data una direzione $\underline{v}$ si definisce **DERIVATA DIREZIONALE**, **SE ESISTE**, il limite del rapporto incrementale in quella direzione, ovvero:

$$\lim_{t \to 0} \frac{f(\underline{x}_0 + \underline{v}t) - f(\underline{x}_0)}{t}$$

Se $\underline{v}$ è una direzione della base canonica, allora tale limite, se esiste, si dice **DERIVATA PARZIALE**.
Il gradiente è il vettore dato dalle derivate parziali in tutte le direzioni canoniche:

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_N} \right)$$

4) Descrivi il "metodo iterativo di discesa del gradiente" e la sua finalità.

Sia $f : \mathbb{R}^N \to \mathbb{R}$. Il metodo di **DISCESA DEL GRADIENTE** è un approccio iterativo per la ricerca di eventuali **MINIMI** della funzione f.
Sia $\underline{x}_0 \in \mathbb{R}^N$ nel dominio di f. Ad ogni passo, esso viene "spostato" nella **DIREZIONE OPPOSTA** rispetto a quella individuata dal gradiente ∇f, per un fattore moltiplicativo detto **LEARNING RATE**.

$$\begin{cases} \underline{x}_0 = \underline{a} \in \mathbb{R}^N & \text{arbitrario} \\ \underline{x}_t = \underline{x}_{t-1} - \eta \nabla f(\underline{x}_{t-1}) \end{cases}$$

5) Descrivi il metodo della discesa del gradiente. In quali casi potrebbe non convergere al minimo globale?

IDEM come sopra + Non è garantita la **CONVERGENZA**, ad esempio, per $N = 1$:

[Grafico: Viene mostrato il grafico di una funzione su un piano cartesiano con diversi avvallamenti che rappresentano minimi locali e globali. L'asse orizzontale è il dominio, l'asse verticale è il valore della funzione.

- Il primo punto di partenza x_0 (indicato in arancione) si trova su un pendio; le frecce blu mostrano la discesa verso un punto indicato come **MINIMO LOCALE**. Un commento spiega che x_0 è troppo vicino al minimo locale e si "incastra" senza raggiungere l'ottimo.
- Il secondo punto di partenza $\tilde{x}_0$ si trova in una posizione più favorevole per raggiungere il **MINIMO GLOBALE**, ma le frecce si fermano prima di arrivarci. Un commento spiega che il numero di iterazioni è insufficiente.
- Sono indicati graficamente i punti critici: Minimo Locale e Minimo Globale.]

Il punto x_0 è troppo vicino a un minimo **LOCALE** e si "incastra" senza raggiungere l'ottimo.
Il punto $\tilde{x}_0$ è in una buona posizione ma il numero di iterazioni è insufficiente.
Altri casi: learning rate troppo basso $\to$ il metodo ci mette troppo tempo; oppure LR troppo alto $\to$ instabilità e oscillazioni; ecc...

6) Come funziona un Single Layer Perceptron (SLP)? Quali sono i suoi limiti?

SLP è un **GRAFO ORIENTATO ACICLICO** e **BIPARTITO** i cui vertici sono **NEURONI ARTIFICIALI**. Consiste di uno **STRATO di INPUT** e uno di **OUTPUT**.

[Schema: Un diagramma di una rete neurale semplice (Single Layer Perceptron). A sinistra ci sono N neuroni di input (vettore $\underline{x}$), a destra M neuroni di output (vettore $\underline{y}$). Ogni neurone di input è collegato a ogni neurone di output tramite archi direzionati. Sotto i due gruppi di neuroni sono indicate le quantità N e M con la dicitura "# di neuroni".]

Su ogni arco (i, j) è presente un **PESO** w_{ij} e su ogni neurone di out. è presente un **BIAS** b_j.
Approssima una funzione $F : \mathbb{R}^N \to \mathbb{R}^M$ data da

$$F(\underline{x}) = \sigma(W^T \underline{x} + \underline{b})$$

SLP è un approssimatore **LINEARE**, non approssima tutte le fz **CONTINUE**.

7) Confronta le funzioni di attivazione ReLU, sigmoide e tanh in termini di proprietà matematiche e applicazioni.

ReLU:

$$\sigma(x) = \max(0, x) = \begin{cases} 0 & \text{se } x \leq 0 \\ x & \text{se } x > 0 \end{cases}$$

[Grafico: Mostra la funzione ReLU su un piano cartesiano. La funzione è piatta (zero) per $x \leq 0$ e diventa una retta diagonale con pendenza 1 per $x > 0$.]

$$\sigma \in \mathcal{C}^0(\mathbb{R}), \quad \sigma \nearrow$$

$$\sigma'(x) = \begin{cases} 0 & \text{se } x \leq 0 \\ 1 & \text{se } x > 0 \end{cases}$$

In genere la **ReLU** viene applicata nelle reti neurali moderne, in particolare dopo i layer convoluzionali nelle **CNN**.

SIGMOIDE: $\sigma \in \mathcal{C}^\infty(\mathbb{R}), \quad \sigma \nearrow, \quad |\sigma| \leq 1$

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

[Grafico: Mostra la classica curva a "S" della sigmoide che asintoticamente va da 0 a 1, incrociando l'asse delle ordinate a 0.5.]

$$\sigma'(x) = \sigma(x)(1-\sigma(x))$$

Si usa negli strati lineari se si vuole un output tra 0 e 1, interpretabile come probabilità (es. di appartenere a una determinata classe).

Tanh: $\sigma \in \mathcal{C}^\infty(\mathbb{R}), \quad \sigma \nearrow, \quad |\sigma| \leq 1$

$$\sigma(x) = \tanh(x)$$

[Grafico: Mostra la funzione tangente iperbolica, una curva a "S" simile alla sigmoide ma che spazia tra -1 e 1, passando per l'origine (0,0).]

Simile alla sigmoide come uso in layer lineari, ma spesso usata nelle reti **Ricorrenti** o **LSTM**.

8) Spiega il principio dell'algoritmo di backpropagation in un MLP con un layer nascosto.

L'**algoritmo di BACKPROPAGATION** è una tecnica usata nell'addestramento di **RETI NEURALI** per il **CALCOLO del GRADIENTE** della funzione di costo (o loss).

MLP con 1 strato nascosto:

$$F(\underline{x}) = \underbrace{\sigma(W_2^T \overbrace{\sigma(W_1^T \underline{x} + \underline{b}_1)}^{\underline{s}_1} + \underline{b}_2)}_{\underline{s}_2}$$

$$= \sigma(\underline{s}_2)$$

LOSS: $\mathcal{L}(\Theta) = \mathcal{L}(W_1, W_2, \underline{b}_1, \underline{b}_2)$

Partendo dall'errore sull'ultimo strato:

$$\delta^{(2)} = (\sigma(\underline{s}_2) - \underline{y})\sigma'(\underline{s}_2)$$

Si propaga all'**INDIETRO** sul primo:

$$\delta_j^{(1)} = \sum_k \delta_k^{(2)} w_{jk}^{(1)} \sigma'(\underline{s}_1)$$

(Nota in rosso con freccia: Questo si può spiegare a parole.)

Da cui si ricava:

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} = \frac{1}{n} \sum_{k=1}^{n} \delta^{(l)} \cdot \sigma(\underline{s}^{(l-1)})$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{n} \sum_{k=1}^{n} \delta^{(l)}$$

**somma sugli n elementi del DATASET*

9) Cos'è una funzione di costo e quale ruolo svolge nel processo di apprendimento? Fai un esempio con la MSE.

La **Loss** è una funzione definita su un **DATASET** $D = \{(\underline{x}, \underline{y}) : \underline{x} \in \mathbb{R}^N, \underline{y} \in \mathbb{R}^M\}$
$\mathcal{L} : D \to \mathbb{R}$ che misura l'errore tra i dati attesi e la previsione del modello.

MSE:

$$\mathcal{L}_2(D) = \frac{1}{2n} \sum_{i=1}^{n} \| \underbrace{\underline{y}_i}_{\text{dal DATASET}} - \underbrace{F_\theta(\underline{x}_i)}_{\text{MODELLO}} \|^2$$

dove $|D| = n$.

Essa si può vedere in funzione dei **PARAMETRI** del modello:

$$\mathcal{L}(D) = \mathcal{L}(\Theta)$$

e addestrare il modello diventa così un problema di **MINIMIZZAZIONE della LOSS** al variare dei **PARAMETRI** Θ.

10) Qual è il significato del teorema di approssimazione universale per le reti neurali?

Il **TEOREMA di APPROSSIMAZIONE UNIVERSALE** stabilisce le capacità espressive di una rete neurale.
Sia:

- $\sigma : \mathbb{R} \to \mathbb{R} \quad , \quad \sigma \in \mathcal{C}^1(\mathbb{R}) \quad e \quad \sigma \nearrow$ **FUNZIONE di ATTIVAZIONE**
- $f : K \subseteq \mathbb{R}^N \to \mathbb{R}^M$ dove K è chiuso e limitato (COMPATTO) e $f \in \mathcal{C}^0(K)$ **FUNZIONE OBIETTIVO**

Fissato un margine d'errore $\varepsilon > 0$, allora **ESISTONO**:

- $m_1 \in \mathbb{N}$ # neuroni I strato
- $W^{(1)} \in \mathbb{R}^{N \times m_1} \quad e \quad W^{(2)} \in \mathbb{R}^{m_1 \times M}$ [MATRICI di PESI]
- $b^{(1)} \in \mathbb{R}^{m_1}$ [VETTORE di BIAS del I strato]

t.c. la funzione calcolata da un **MLP** semplificato (con 1 strato nascosto) data da:

$$F(\underline{x}) := {W^{(2)}}^T \cdot \sigma({W^{(1)}}^T \underline{x} + \underline{b}^{(1)})$$

soddisfa la disuguaglianza:

$$\max_{\underline{x}\in K} \|F(\underline{x}) - f(\underline{x})\| < \varepsilon$$

Il **SIGNIFICATO** fondamentale del teorema risiede nel fatto che **NON è un teorema COSTRUTTIVO** $\longrightarrow \exists$ parametri ottimali per approssimare la funzione MA NON so quali sono

11) Quali sono i componenti fondamentali di un "percettrone" o "neurone artificiale", e a cosa servono i "pesi" e il "bias"?

Il **PERCETTRONE** (o **NEURONE ARTIFICIALE**) è il modello base di calcolo nelle reti neurali.
I suoi **COMPONENTI FONDAMENTALI** sono:

- $\underline{x} \in \mathbb{R}^N$ [VETTORE di INPUT]
- $\underline{w} \in \mathbb{R}^N$ [VETTORE dei PESI]
- $b \in \mathbb{R}$ [BIAS]
- $\sigma : \mathbb{R} \to \mathbb{R}$ **FUNZIONE di ATTIVAZIONE** (affinché si parli di neurone artificiale, σ deve essere almeno continua, $\sigma \in \mathcal{C}^0(\mathbb{R})$)

Il modello calcola la funzione $f : \mathbb{R}^N \to \mathbb{R}$ data da:

$$f_{\underline{w},b}(\underline{x}) = \sigma(\underline{x} \cdot \underline{w} + b)$$

FUNZIONE DEI PARAMETRI:
Ciò che DETERMINA l'output è l'argomento $\boxed{\underline{x} \cdot \underline{w} + b}$ $\longrightarrow$ detta **PRE-ATTIVAZIONE**.

- I **PESI** ($\underline{w}$) modulano l'importanza dei singoli segnali di input x_i all'interno della combinazione lineare.
- Il **BIAS** (b) agisce come parametro di traslazione rispetto a una soglia prestabilita T (posto $b = -T$). Determina la propensione all'attivazione del neurone $\implies$ se $\underline{x} \cdot \underline{w} + b > 0 \longrightarrow \text{ATTIVO}$, altrimenti $\leq 0 \longrightarrow \text{NON ATTIVO}$.

12) Definisci la "funzione di costo" e illustra la differenza tra "loss L1" (geometria del taxi) e "loss L2" (MSE).

Def: Si dice **FUNZIONE di COSTO** o di **PERDITA** o **LOSS** una funzione definita su un **DATASET** $D = \{(\underline{x}, y) : \underline{x} \in \mathbb{R}^N, y \in \mathbb{R}^M\}$. Ovvero:

$$\mathcal{L} : D \to \mathbb{R}$$

Essa rappresenta il modo per misurare l'**ERRORE** o il **COSTO** compiendo una minimizzazione tra i dati attesi (funzione obiettivo y^*) e la previsione calcolata dal **MODELLO** ($F_\theta(\underline{x})$).

La differenza tra le due metriche presentate è la seguente:

- **LOSS L_1:**

  $$\mathcal{L}_1(D) := \sum_{i=1}^{m} \| \underbrace{y_i^*}_{\text{dal DATASET}} - \underbrace{F_\theta(\underline{x}_i)}_{\text{MODELLO}} \|$$

  dove $|D| = m$. Geometricamente e visualmente, la differenza tra le coordinate viene calcolata sommando gli scarti assoluti lungo percorsi a gradini, definendo la cosiddetta **GEOMETRIA del TAXI**.
- **LOSS $\mathcal{L}_2$, ERRORE QUADRATICO MEDIO, MEAN-SQUARED ERROR (MSE):**

  $$\mathcal{L}_2(D) = \frac{1}{2m} \sum_{i=1}^{m} \| \underbrace{y_i^*}_{\text{dal DATASET}} - F_\theta(\underline{x}_i)\|^2$$

  Differisce da $\mathcal{L}_1$ in quanto quantifica l'errore tramite l'elevamento al quadrato della norma euclidea e media il risultato applicando il termine $\frac{1}{2m}$.

13.) Elenca e descrivi brevemente almeno quattro "funzioni di attivazione" comunemente utilizzate nelle reti neurali artificiali.

Le **FUNZIONI di ATTIVAZIONE** ($\sigma : \mathbb{R} \to \mathbb{R}$) introducono non-linearità nei modelli. Per le reti neurali artificiali, le funzioni comunemente utilizzate sono non decrescenti ($\sigma \nearrow$) e richiedono almeno la continuità ($\sigma \in \mathcal{C}^0(\mathbb{R})$).

Eccone quattro tra le principali:

1) SIGMOIDE LOGISTICA: $\sigma \in \mathcal{C}^\infty(\mathbb{R}), \quad \sigma \nearrow, \quad |\sigma| \leq 1$

$$\sigma_\beta(x) = \frac{1}{1 + e^{-\beta x}}$$

La sua derivata prima è data da:

$$\frac{d\sigma}{dx}(x) = \beta\sigma(x)(1 - \sigma(x))$$

2) RECTIFIED LINEAR UNIT (ReLU): $\sigma \in \mathcal{C}^0(\mathbb{R}), \quad \sigma \nearrow$

$$\sigma(x) = \max\{0, x\} = \begin{cases} 0 & \text{se } x \leq 0 \\ x & \text{altrimenti} \end{cases}$$

La cui derivata è a gradino:

$$\frac{d\sigma}{dx}(x) = \begin{cases} 0 & \text{se } x \leq 0 \\ 1 & \text{se } x > 0 \end{cases}$$

3) SOFTPLUS (SMOOTH ReLU): $\sigma \in \mathcal{C}^\infty(\mathbb{R}), \quad \sigma \nearrow$

$$\sigma_\beta(x) = \frac{1}{\beta}\log(1 + e^{\beta x})$$

Costituisce una versione differenziabile ("smussata") della ReLU standard.

4) TANGENTE IPERBOLICA (Tanh): $\sigma \in \mathcal{C}^\infty(\mathbb{R}), \quad \sigma \nearrow, \quad |\sigma| \leq 1$

$$\sigma_\beta(x) = \tanh(\beta x) = \frac{e^{\beta x} - e^{-\beta x}}{e^{\beta x} + e^{-\beta x}} = \frac{1 - e^{-2\beta x}}{1 + e^{-2\beta x}}$$

14) Qual è il principio su cui si basa l'algoritmo di "backpropagation" e come mai è importante per l'addestramento di MLP?

L'**algoritmo di BACK PROPAGATION** è una tecnica usata nell'addestramento di **RETI NEURALI** che si basa sull'applicazione della **CHAIN-RULE** (derivata della funzione composta) per il **CALCOLO del GRADIENTE** della funzione di costo (o loss).

L'**OBIETTIVO** è calcolare in modo efficiente il gradiente $\nabla_\theta \mathcal{L}$, ovvero le derivate parziali rispetto a tutti i parametri del modello (matrici di pesi W e vettori di bias $\underline{b}$):

$$\nabla_\theta \mathcal{L} \longrightarrow \begin{cases} \frac{\partial \mathcal{L}}{\partial W^{(1)}} & \frac{\partial \mathcal{L}}{\partial \underline{b}^{(1)}} \\ \dots & \dots \\ \frac{\partial \mathcal{L}}{\partial W^{(L)}} & \frac{\partial \mathcal{L}}{\partial \underline{b}^{(L)}} \end{cases}$$

PRINCIPIO DI FUNZIONAMENTO:
Partendo dall'**ERRORE sullo STRATO di OUTPUT**, esso viene valutato calcolando la differenza tra la previsione e il valore reale:

$$\delta_k^{(L+1)} = (\underbrace{\sigma(s_k^{(L+1)})}_{\text{CALCOLATO}} - \underbrace{y_k}_{\text{dal DATASET}}) \cdot \sigma'(s_k^{(L+1)})$$

Successivamente, l'errore si propaga all'**INDIETRO** andando a calcolare gli **ERRORI negli STRATI NASCOSTI**:

$$\delta_j^{(l)} = \sum_{k=1}^{m_{l+1}} \delta_k^{(l+1)} \cdot w_{jk}^{(l)} \cdot \sigma'(s_j^{(l)})$$

Da queste quantità si ricavano le **EQUAZIONI della BACK-PROPAGATION** definitive per calcolare il gradiente rispetto a ciascun peso e bias (da usare poi nella discesa del gradiente):

$$\frac{\partial \mathcal{L}}{\partial w_{ij}^{(l)}} = \frac{1}{m} \sum_{k=1}^{m} \delta_j^{(l)} \cdot \sigma(s_i^{(l-1)}) \qquad \frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \frac{1}{m} \sum_{k=1}^{m} \delta_j^{(l)}$$

IMPORTANZA per gli MLP:
L'algoritmo è di importanza cruciale per l'addestramento degli **MLP** perché permette di determinare l'**ERRORE sul NEURONE interno j-ESIMO**. Negli strati nascosti, a differenza dell'ultimo strato, non esiste un valore y^* fornito dal dataset con cui confrontare l'output del neurone. La backpropagation risolve questo problema calcolando quanto ciascun parametro interno abbia contribuito all'errore finale compiuto dal modello, rendendo così possibile l'ottimizzazione iterativa.

15) Cosa afferma il "Teorema di approssimazione universale"?

Il **TEOREMA di APPROSSIMAZIONE UNIVERSALE** afferma che, date:

- $\sigma : \mathbb{R} \to \mathbb{R} \quad , \quad \sigma \in \mathcal{C}^1(\mathbb{R}) \quad e \quad \sigma \nearrow$ **FUNZIONE di ATTIVAZIONE**
- $f : K \subseteq \mathbb{R}^N \to \mathbb{R}^M$ dove K è chiuso e limitato (COMPATTO) e $f \in \mathcal{C}^0(K)$ **FUNZIONE OBIETTIVO**

Fissato un margine di errore $\varepsilon > 0$, allora **ESISTONO**:

- $m_1 \in \mathbb{N}$ # neuroni I strato
- $W^{(1)} \in \mathbb{R}^{N \times m_1} \quad e \quad W^{(2)} \in \mathbb{R}^{m_1 \times M}$ [MATRICI di PESI]
- $b^{(1)} \in \mathbb{R}^{m_1}$ [VETTORE di BIAS del I strato]

t.c. la funzione calcolata da un **MLP** semplificato (con 1 strato nascosto) data da:

$$F(\underline{x}) := {W^{(2)}}^T \cdot \sigma({W^{(1)}}^T \underline{x} + \underline{b}^{(1)})$$

soddisfa la disuguaglianza:

$$\max_{\underline{x} \in K} \|F(\underline{x}) - f(\underline{x})\| < \varepsilon$$

OSS: Il teorema garantisce che la rete può approssimare la funzione con precisione arbitraria, ma è un **teorema NON COSTRUTTIVO** $\longrightarrow \exists$ parametri ottimali MA NON so quali sono.

16) Descrivere le fasi principali del processo di "digitalizzazione delle immagini", specificando i concetti di "campionamento" e "quantizzazione".

Il processo di **DIGITALIZZAZIONE delle IMMAGINI** converte una funzione continua $f : \mathbb{R}^2 \longrightarrow \mathcal{C}$ (dove $\mathcal{C}$ è lo **SPAZIO COLORIMETRICO**) in un formato discreto. Consiste in due fasi principali:

- **CAMPIONAMENTO:**
  Scompone l'immagine in **PIXEL** creando una griglia spaziale $\underbrace{H \times W}_{\text{RISOLUZIONE}}$.
- **QUANTIZZAZIONE:**
  Dipende dallo spazio colorimetrico $\mathcal{C}$ scelto (es. **RGB** $\subseteq \mathbb{R}^3$ per Rosso, Verde, Blu; **HSV** $\subseteq \mathbb{R}^3$ per Tonalità; **CMYK** $\subseteq \mathbb{R}^4$).
  Ha 2 parametri fondamentali:
  - N_B : numero di **BIT**
  - N_C : numero di **COLORI** da rappresentare

  La relazione che lega i bit ai colori è:

$$2^{N_B} \geq N_C \iff N_B \geq \log_2 N_C \rightsquigarrow N_B = \lceil \log_2 N_C \rceil$$

17) Cosa si intende per immagine digitale? Spiega i concetti di campionamento, quantizzazione e spazio colorimetrico.

Def: Si definisce **IMMAGINE** una funzione continua $f : \mathbb{R}^2 \longrightarrow \mathcal{C}$, dove $\mathcal{C}$ è detto **SPAZIO COLORIMETRICO** (con $\mathcal{C} \subseteq \mathbb{R}^3$ oppure $\mathcal{C} \subseteq \mathbb{R}^4$).

Il passaggio a un' **IMMAGINE DIGITALIZZATA** (formalizzata matematicamente come un **TENSORE**) si articola in due fasi principali:

- **CAMPIONAMENTO:**
  Agisce sul dominio spaziale. Scompone l'immagine in **PIXEL** creando una griglia discreta determinata dalla $\underbrace{H \times W}_{\text{RISOLUZIONE}}$.
- **QUANTIZZAZIONE e SPAZIO COLORIMETRICO:**
  Agisce sul codominio e dipende dallo spazio colorimetrico $\mathcal{C}$ scelto per la codifica. I principali spazi sono:

1. **RGB** $\subseteq \mathbb{R}^3$ (Rosso, Verde, Blu)
2. **HSV** $\subseteq \mathbb{R}^3$ (Hue/Tonalità, Saturation, Value)
3. **CMYK** $\subseteq \mathbb{R}^4$ (Cyan, Magenta, Yellow, Black)

L'operazione di quantizzazione è governata da 2 parametri:

- N_B : numero di **BIT**
- N_C : numero di **COLORI** da rappresentare

La relazione fondamentale che li lega è:

$$2^{N_B} \geq N_C \iff N_B \geq \log_2 N_C \rightsquigarrow N_B = \lceil \log_2 N_C \rceil$$

18) Come si utilizza un istogramma per operazioni di elaborazione su immagini in scala di grigi?

Def: Si definisce **ISTOGRAMMA dei LIVELLI di GRIGIO** una funzione:

$$h : \{0, 1, \dots, 2^{N_B} - 1\} \longrightarrow \mathbb{N}$$

dove N_B è il numero di **BIT** usati nella quantizzazione dell'immagine. Esso quantifica la frequenza di ciascun livello di grigio ℓ nell'immagine:

$$h(\ell) = |\{L_{ij} = \ell, i = 1, \dots, H, j = 1, \dots, W\}|$$

Nell'ambito delle **IMMAGINI LUMINANZA** (o in **SCALA di GRIGI**), l'istogramma si utilizza per effettuare diverse operazioni:

- **SOGLIATURA (THRESHOLDING):** Fissato un valore **SOGLIA** T, è possibile isolare gli elementi dell'immagine (es. separare un oggetto chiaro da uno sfondo scuro) definendo $t(\ell) = |\{h(\ell) \geq T\}|$. Da questo si può ricavare l'**AREA** dell'oggetto:

$$\text{AREA} = \sum_{\ell=0}^{2^{N_B}-1} t(\ell)$$

- **OPERATORI PUNTUALI** $\longrightarrow$ **TRASFORMAZIONI:** L'istogramma permette di applicare trasformazioni dirette sui livelli di grigio mappando l'insieme originale in un nuovo insieme trasformato:

$$\{h_0, h_1, h_2, \dots, h_{2^{N_B}-1}\} \overset{\sigma}{\Longrightarrow} \{\hat{h}_0, \hat{h}_1, \dots, \hat{h}_{2^{N_B}-1}\}$$

  Questo principio è alla base di operazioni volte a migliorare o modificare la resa visiva, quali l'**INVERSIONE**, la **COMPRESSIONE LOGARITMICA**, la **COMPRESSIONE POTENZA** e l'**ESPANSIONE di CONTRASTO**.

19) Spiega come funziona il filtro di Sobel e quale informazione estrae da un'immagine.

Il **FILTRO di SOBEL** è un operatore locale differenziabile del **I ORDINE** con dimensione 3×3, utilizzato come filtro di **SHARPENING**.
Si basa sull'approssimazione delle derivate parziali spaziali dell'immagine tramite i **POLINOMI DI TAYLOR**. Il filtro agisce applicando due **MASCHERE** (o **KERNEL**) $M_x, M_y \in \mathbb{R}^{3\times3}$:

$$M_x = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} \qquad M_y = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

Data un'immagine luminanza L, le derivate parziali vengono calcolate applicando l'operazione di convoluzione tra l'immagine e le rispettive maschere:

- $L_x = L * M_x \quad \longrightarrow$ derivata parziale rispetto a x
- $L_y = L * M_y \quad \longrightarrow$ derivata parziale rispetto a y

INFORMAZIONE ESTRATTA:
Il filtro serve a estrarre e delineare i **CONTORNI (EDGES)**, evidenziando le zone in cui vi è una forte variazione di intensità luminosa. L'output finale si ottiene calcolando l'ampiezza globale del gradiente ∇L, definendo il cosiddetto **FILTRO GRADIENTE** G:

$$G = |L_x| + |L_y| \qquad \text{oppure} \qquad \|\nabla L\| = \sqrt{\left(\frac{\partial L}{\partial x}\right)^2 + \left(\frac{\partial L}{\partial y}\right)^2}$$

20) Qual è la differenza tra convoluzione e cross-correlazione? Fornisci una definizione formale.

Def: Si definisce **OPERAZIONE di CONVOLUZIONE** tra un SEGNALE in INPUT x e un NUCLEO o **KERNEL** w l'operazione data da:

- $N = 1$**:**

$$(x * w)(t) = \int_{\mathbb{R}} x(a) \cdot w(t - a) da$$

- $N = 2$**:**

$$(x * w)(u, v) = \int_{\mathbb{R}^2} x(a, b) w(u - a, v - b) da db$$

Def: Si dice **CORRELAZIONE INCROCIATA (CROSS-CORRELATION)** di un segnale in INPUT x contro un KERNEL w l'operazione definita come:

$$(x \star w)(t) = \int_{\mathbb{R}} x(a) w(t + a)\, da$$

DIFFERENZA FONDAMENTALE:
La differenza matematica tra le due operazioni risiede nell'argomento del kernel all'interno dell'integrale. Nella convoluzione il kernel viene invertito rispetto alla variabile di integrazione ($-a$), mentre nella cross-correlazione il nucleo mantiene lo stesso segno ($+a$).
Come diretta conseguenza, calcolare la cross-correlazione equivale a effettuare una convoluzione utilizzando un **KERNEL FLIPPATO** $w^-(t) = w(-t)$:

$$(x \star w)(t) = (x * w^-)(t)$$

21) Quali sono le proprietà principali dell'operazione di convoluzione?

L'**operazione di CONVOLUZIONE** tra un segnale di input x e un kernel w gode di tre **PROPRIETÀ PRINCIPALI**:

1. COMMUTATIVA:

$$(x * w) = (w * x)$$

2. LINEARITÀ:
Eredita la linearità dell'operatore di integrazione. Dati due segnali in input x, y e costanti $\lambda, \mu \in \mathbb{R}$:

$$(\lambda x + \mu y) * w = \lambda(x * w) + \mu(y * w)$$

3. INVARIANZA ALLE TRASLAZIONI:
L'operazione riflette esattamente nel risultato eventuali traslazioni applicate al segnale di input. Si declina in due modi a seconda delle dimensioni:

- **Per $N = 1$ (Invarianza rispetto al TEMPO t):**
  Considerando un segnale $\tilde{x} : \mathbb{R} \to \mathbb{R}$ ritardato di τ rispetto a x, ovvero $\tilde{x}(t) = x(t - \tau)$:

$$(\tilde{x} * w)(t) = (x * w)(t - \tau)$$

- **Per $N = 2$ (Indipendenza dalla POSIZIONE):**
  Considerando un segnale $\tilde{x} : \mathbb{R}^2 \to \mathbb{R}$ traslato spazialmente di (u_0, v_0), ovvero $\tilde{x}(u, v) = x(u - u_0, v - v_0)$:

$$(\tilde{x} * w)(u, v) = (x * w)(u - u_0, v - v_0)$$

22) Illustra l'utilizzo dell'"istogramma dei livelli di grigio" in relazione agli "operatori puntuali" per la manipolazione di immagini luminanza.

L'**ISTOGRAMMA dei LIVELLI di GRIGIO** $h(\ell)$ descrive la frequenza dei livelli di grigio in un'immagine luminanza. Gli **OPERATORI PUNTUALI** agiscono su di esso applicando delle **TRASFORMAZIONI** dirette che mappano l'insieme originale in un nuovo insieme trasformato:

$$\{h_0, h_1, \dots, h_{2^{N_B}-1}\} \overset{\sigma}{\Rightarrow} \{\hat{h}_0, \hat{h}_1, \dots, \hat{h}_{2^{N_B}-1}\}$$

Le principali operazioni (definite per un'immagine discreta $L \in \mathbb{R}^{H \times W}$ con intensità massima N_ℓ) sono:

- **INVERSIONE:**

$$I_{ij} = N_\ell - L_{ij}$$

- **COMPRESSIONE LOGARITMICA:** (Espande i livelli bassi e comprime i livelli alti)

$$Y_{ij} = \frac{1}{1 + N_\ell} \log(1 + L_{ij})$$

- **COMPRESSIONE POTENZA** (con parametro $\gamma > 0$):

$$P_{ij} = N_\ell^{1-\gamma} \cdot L_{ij}^{\gamma}$$

  (Se $\gamma \in (0, 1)$ simula la compressione logaritmica; se $\gamma = 1$ l'immagine resta inalterata; se $\gamma > 1$ comprime gli scuri).
- **ESPANSIONE di CONTRASTO** (fissata una soglia T):

$$Y_{ij} = \begin{cases} 0 & \text{se } L_{ij} < T \\ N_\ell \cdot \frac{L_{ij} - T}{N_\ell - T} & \text{se } L_{ij} \geq T \end{cases}$$

IMPLEMENTAZIONE: Queste trasformazioni puntuali vengono tipicamente implementate in modo efficiente tramite **LOOKUP-TABLES (LUT o COLORMAP)**, ovvero array che mappano gli indici di input $(0 \div N_\ell)$ nei rispettivi valori di output con un costo computazionale di $O(1)$.

23) In che modo i "polinomi di Taylor" vengono applicati per la creazione di "filtri di sharpening" come il filtro di Sobel o il filtro Laplaciano?

Il calcolo delle derivate spaziali (es. ∇I) su un'**IMMAGINE DIGITALIZZATA** (che è discreta) costituisce un problema che viene risolto tramite l'approssimazione con i **POLINOMI DI TAYLOR**.

Preso un **PIXEL centrale** $\underline{x}_0 = (x, y)$ e un punto nel suo **INTORNO** $\underline{x} = (x + h, y + k)$ con $h, k \in \{-1, 0, 1\}$, lo sviluppo di Taylor fino al II ordine è:

$$f(x + h, y + k) \cong f(x, y) + \frac{\partial f}{\partial x} h + \frac{\partial f}{\partial y} k + \frac{\partial^2 f}{\partial x^2} \frac{h^2}{2} + \frac{\partial^2 f}{\partial y^2} \frac{k^2}{2} + 2 \frac{\partial f}{\partial x \partial y} \frac{hk}{2}$$

A partire da questa formula generale, si ricavano i pesi dei **FILTRI di SHARPENING**:

- **FILTRO di SOBEL (Operatore del I ORDINE):**
  Troncando l'approssimazione al primo ordine, si possono isolare le stime delle derivate parziali. Ad esempio, per la direzione verticale ($k = 0, h = \pm 1$):

$$\frac{\partial f}{\partial x}(x, y) \cong \frac{f(x + 1, y) - f(x - 1, y)}{2}$$

  Combinando sistematicamente queste approssimazioni (orizzontali, verticali e diagonali), si costruiscono le **MASCHERE** 3×3 M_x e M_y usate per il calcolo del gradiente.
- **FILTRO LAPLACIANO (Operatore del II ORDINE):**
  Sfrutta lo sviluppo al secondo ordine per approssimare l'operatore $\nabla^2 f := \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$. Ponendo opportunamente h e k a ± 1 e 0, si ottiene:

$$\frac{\partial^2 f}{\partial x^2}(x, y) \cong f(x - 1, y) - 2f(x, y) + f(x + 1, y)$$

$$\frac{\partial^2 f}{\partial y^2}(x, y) \cong f(x, y - 1) - 2f(x, y) + f(x, y + 1)$$

  Dalla somma diretta di questi contributi discreti si ricava la maschera matriciale finale del filtro:

$$M_{\nabla^2} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

24) Definisci l' "operazione di convoluzione" nel continuo e indicane le principali proprietà, sia in una che in due variabili.

Def: Si definisce **OPERAZIONE di CONVOLUZIONE** nel continuo l'operazione che scontra un **SEGNALE IN INGRESSO** x con un **NUCLEO o KERNEL** w.

Si formalizza in base alle dimensioni:

- **Per $N = 1$:** Dati $x : \mathbb{R} \to \mathbb{R}$ e $w : \mathbb{R} \to \mathbb{R}$,

$$(x * w)(t) = \int_{\mathbb{R}} x(a) \cdot w(t-a) da$$

- **Per $N = 2$:** Dati $x : \mathbb{R}^2 \to \mathbb{R}$ e $w : \mathbb{R}^2 \to \mathbb{R}$,

$$(x * w)(u, v) = \int_{\mathbb{R}^2} x(a, b) \cdot w(u - a, v - b)\, da\, db$$

La convoluzione gode di tre **PROPRIETÀ PRINCIPALI** (valide sia in 1 che in 2 variabili):

1. COMMUTATIVA:

$$(x * w) = (w * x)$$

2. LINEARITÀ:
Eredita la linearità dell'operatore di integrazione. Dati due segnali in input x, y e costanti $\lambda, \mu \in \mathbb{R}$:

$$(\lambda x + \mu y) * w = \lambda(x * w) + \mu(y * w)$$

3. INVARIANZA (o Indipendenza dalle Traslazioni):

- $N = 1$ **(Invariante rispetto al TEMPO t):** Considerato un segnale ritardato di τ, ovvero $\tilde{x}(t) = x(t - \tau) \implies (\tilde{x} * w)(t) = (x * w)(t - \tau)$
- $N = 2$ **(Indipendente dalla POSIZIONE):** Considerato un segnale traslato spazialmente, ovvero $\tilde{x}(u, v) = x(u - u_0, v - v_0) \implies (\tilde{x} * w)(u, v) = (x * w)(u - u_0, v - v_0)$

25) Quali sono le differenze strutturali e funzionali tra un MLP e una CNN?

Le principali **DIFFERENZE STRUTTURALI e FUNZIONALI** tra le due architetture sono:

1. GESTIONE DELL'INPUT:

- **MLP:** Gestisce input unidimensionali, prendendo in ingresso un vettore $\underline{x} \in \mathbb{R}^N$ di dimensione N fissata.
- **CNN:** È progettata specificamente per gestire dati a "GRIGLIA" (come le immagini digitalizzate), ragionando direttamente a **VOLUMI** o **TENSORI**. Prende in input $I \in \mathbb{R}^{H \times W \times c}$ dove altezza, larghezza e canali sono fissati.

2. STRUTTURA DELLE CONNESSIONI:

- **MLP:** Utilizza strati **FULLY CONNECTED** (densamente connessi), in cui ogni neurone elabora tutti gli elementi del layer precedente tramite un'intera matrice di pesi W.
- **CNN:** Si basa su blocchi convoluzionali che applicano piccoli filtri/kernel (es. di dimensione $n \times n$) in grado di spostarsi spazialmente sull'input.

3. NUMERO DI PARAMETRI (PESI CONDIVISI):

- A differenza dell'MLP, che necessita di addestrare un peso diverso per ogni connessione, la **CNN** si basa sul principio dei **PESI CONDIVISI** con tutti i pixel.
- Dal punto di vista funzionale, questo abbatte drasticamente il costo computazionale: ad esempio, se il filtro ha dimensione $n = 3$, la CNN avrà **SOLO 9 PARAMETRI** da addestrare per produrre una determinata *slice* di output, rendendo la rete enormemente più efficiente.

26) Perché le "Convolutional Neural Networks (CNN)" sono particolarmente efficaci per la gestione di "dati a griglia", come le immagini?

Le **CONVOLUTIONAL NEURAL NETWORKS (CNN)** sono particolarmente efficaci per la gestione di dati a "GRIGLIA" (come le immagini) grazie alle seguenti caratteristiche strutturali:

- **ELABORAZIONE a VOLUMI (TENSORI):** A differenza degli MLP, le CNN mantengono intatta la struttura spaziale originaria ragionando direttamente su input multidimensionali $I \in \mathbb{R}^{H \times W \times c}$ (dove H, W indicano la risoluzione spaziale e c il numero di canali, es. RGB).
- **FILTRI LOCALI (KERNEL):** Utilizzano **BLOCCHI CONVOLUZIONALI** in cui non tutti i neuroni sono connessi a tutto l'input. Vengono invece applicati piccoli filtri spaziali di dimensione $n \times n$ (con n dispari) che scorrono sull'immagine a passi definiti dal parametro **STRIDE**.
- **PESI CONDIVISI:** È il principio chiave dell'efficienza delle CNN. Lo stesso filtro $n \times n$ viene fatto scorrere e applicato a tutti i pixel della griglia mantenendo esattamente gli stessi pesi.
  $\longrightarrow$ **VANTAGGIO COMPUTAZIONALE:** Questo meccanismo abbatte drasticamente il numero di parametri da ottimizzare. Ad esempio, se si fissa un filtro 3×3, la rete avrà **SOLO 9 PARAMETRI** da addestrare per generare una determinata *slice* (mappa di caratteristiche) dell'output, a prescindere dalla grandezza totale dell'immagine in input.

27) Dare una breve descrizione dei minimi esempi di strati utilizzati in una CNN: "layer convoluzionale", "funzione di attivazione ReLU", "layer di pooling", e "dropout".

La struttura standard di una **CNN** prevede il susseguirsi di diversi strati (layers):

- **LAYER CONVOLUZIONALE:** Elabora dati a "griglia" ragionando direttamente a volumi/tensori. I suoi **IPERPARAMETRI** fondamentali sono:
  - **DEPTH** (d): profondità o numero di filtri.
  - **STRIDE** (s): passo di spostamento del filtro.
  - **ZERO-PADDING** (p): dimensione del contorno di zeri aggiunto all'input.
  - **DIMENSIONE KERNEL:** $n \times n$ (con n dispari).
    Si basa sul principio dei **PESI CONDIVISI** (lo stesso filtro scorre su tutti i pixel) che permette di abbattere drasticamente i parametri da addestrare.
- **FUNZIONE di ATTIVAZIONE ReLU (Rectified Linear Unit):**
  Applicata tipicamente in uscita dai blocchi convoluzionali per introdurre non-linearità.

$$\sigma(x) = \max\{0, x\} = \begin{cases} 0 & \text{se } x \leq 0 \\ x & \text{altrimenti} \end{cases}$$

- **LAYER di POOLING:** Costituisce un filtro **NON-LINEARE**. Il metodo standard è il **MAX-POOLING**, il quale possiede una dimensione $n \times n$ e uno stride s. Fa scorrere una finestra sull'input e restituisce esclusivamente il valore massimo dell'intorno:

$$x_{ij} \longmapsto \max(x_{i+m, j+m}) = p_{ij}^k$$

  (NB: L'operazione "max", non essendo strettamente differenziabile, richiede un trattamento particolare - i valori non massimi annullano il gradiente - durante la Backpropagation).
- **DROPOUT:**
  (Nota: Nei documenti e negli appunti forniti non vi è alcuna nozione, formula o riferimento alla tecnica del Dropout. Si consiglia di consultare testi o manuali esterni per completare questo punto, in quanto tale concetto non è presente nelle fonti analizzate).

28) Per quale tipo di dati sono concepite le "Recurrent Neural Networks (RNN)" e quale problematica affrontano?

Le **RETI NEURALI RICORRENTI (RNN)** sono concepite specificamente per elaborare il **DATO SEQUENZIALE**, come ad esempio musica, video, serie temporali o testo.

A differenza dei modelli tradizionali in cui la dimensionalità è fissa:

- **MLP:** riceve un input unidimensionale $\underline{x} \in \mathbb{R}^N$ con N fissato.
- **CNN:** riceve un volume in input $I \in \mathbb{R}^{H \times W \times c}$ con H, W e canali c fissati.

PROBLEMATICA AFFRONTATA:
Le RNN risolvono la difficoltà di **Gestire la DIMENSIONE VARIABILE** introdotta dal parametro temporale t.
Per farlo, l'architettura introduce dei **CICLI** di feedback: la rete non elabora unicamente l'input corrente al tempo t, ma riceve in ingresso anche il proprio output calcolato al passo temporale precedente, introducendo un ritardo $y(t-1)$.

Srotolando la RNN nel tempo per una sequenza di input $\underline{x}(0), \underline{x}(1), \ldots, \underline{x}(t)$, il calcolo dell'output al generico passo t è governato dall'equazione:

$$\underline{y}(t) = \sigma(W \cdot \underline{x}(t) + \hat{W} \cdot \underline{y}(t-1) + \underline{b})$$

.

In questo modo, la rete combina l'input attuale $\underline{x}(t)$ pesato dalla matrice W, con lo stato passato $\underline{y}(t-1)$ pesato dalla matrice $\hat{W}$, permettendo al modello di mantenere una sorta di "memoria" della sequenza.

29) Quali sono le principali caratteristiche della "funzione di costo cross-entropy" e della "funzione di attivazione softmax", e in quale contesto sono spesso utilizzate?

La **CROSS-ENTROPY** (funzione di costo) e la **SOFTMAX** (funzione di attivazione) sono strettamente legate alla Teoria dell'Informazione e trovano il loro utilizzo principale nell'ambito dei **MODELLI di LINGUAGGIO** (es. NLP, Word Embedding).

- **FUNZIONE DI COSTO CROSS-ENTROPY:**
  Si basa sul concetto di "quantità di sorpresa". Supponendo di avere una distribuzione stimata o nota dei dati ψ, la cross-entropy quantifica la sorpresa nello scoprire che la distribuzione reale è in realtà φ.
  L'obiettivo della rete è minimizzare questa funzione di costo (il che equivale a massimizzare la *Likelihood* o verosimiglianza). Si definisce come:

$$\mathcal{L}(\tilde{I}, I) = -\sum_{i=0}^{m} \varphi(x_i) \log(\psi(x_i))$$

- **FUNZIONE DI ATTIVAZIONE SOFTMAX:**
  È una funzione $\sigma : \mathbb{R}^N \to \mathbb{R}$ utilizzata tipicamente nell'ultimo strato della rete. Serve a mappare i valori in uscita in una distribuzione di probabilità normalizzata. Per la componente x_i, si calcola come:

$$\sigma(x_i) = \frac{\overbrace{e^{x_i}}^{\text{COMPONENTE}}}{\underbrace{\sum_{k=0}^{N} e^{x_k}}_{\text{NORMALIZZAZIONE}}}$$

UTILIZZO CONGIUNTO:
Sostituendo la funzione Softmax all'interno dell'equazione della Cross-Entropy (cioè ponendo $\psi(x_i) = \sigma(x_i)$), si ottiene una forma analitica ottimizzata per il calcolo della *loss* durante l'addestramento, che si riduce a:

$$\boxed{\log\left(\sum_k e^{x_k}\right) - \mathbb{E}_{\varphi}[X]} \quad \longrightarrow \text{CROSS-ENTROPY} + \text{SOFTMAX}$$

30) Descrivi l'approccio dei modelli "Generative Adversarial Networks (GAN)".

L'approccio dei modelli **GENERATIVE ADVERSARIAL NETWORKS (GAN)** si basa sull'addestramento **SIMULTANEO** di due reti neurali distinte che competono tra loro.

- $G \longrightarrow$ **Rete GENERATORE:** $\boxed{g(z, \theta_y)}$. Riceve in input una variabile latente e genera nuovi campioni tramite il percorso $[\underline{x} \to \underline{z} \to y]$.
- $D \longrightarrow$ **Rete DISCRIMINATORE:** È una rete "deterministica" (come MLP o CNN) che effettua una **CLASSIFICAZIONE BINARIA** $\underline{x} \longmapsto d(x, \theta_x)$. Il suo scopo è distinguere se un dato in input è REALE ($x \in D_{\text{train}}$) o FAKE (cioè un dato generato, $g(x)$).

METODO DI ADDESTRAMENTO:
Il processo di addestramento è formulato matematicamente come un **GIOCO a SOMMA ZERO**. Entrambi i giocatori (le reti) mirano a massimizzare il proprio payoff: il payoff di D è dato da $v(\theta_y, \theta_x)$, mentre il payoff di G è dato da $-v(\theta_y, \theta_x)$.

Il problema si risolve tramite l'**ALGORITMO MINIMAX**, con l'obiettivo di trovare $g^* = \arg\min_g \max_d v(g, d)$.
La funzione valore v (che funge da **LOSS AVVERSARIA** legata alla Log-Likelihood) è definita dall'equazione:

$$v(\theta_y, \theta_x) = \mathbb{E}_{x \sim \text{data}}[\underbrace{\log(d(x, \theta_x))}_{P}] + \mathbb{E}_{g(z) \sim \text{model}}[\underbrace{\log(1 - d(g(x), \theta_x))}_{1-P}]$$

CONVERGENZA:
Se la rete arriva a convergenza, si ottiene il generatore ottimale g^* nel momento in cui il discriminatore non sa più distinguere i dati reali da quelli falsi, restituendo $d(x, \theta_x) \approx \frac{1}{2}$ per ogni x. Si noti, tuttavia, che nell'addestramento delle GAN non c'è garanzia teorica di raggiungere tale equilibrio.

31) Quali problemi possono sorgere nella retropropagazione in MLP profondi (vanishing/exploding gradients)?

Il **VANISHING GRADIENT PROBLEM** (o il duale **EXPLODING GRADIENT**) è un problema numerico che si verifica nell'addestramento di reti neurali profonde, in cui i gradienti assumono valori troppo BASSI (vanishing) o troppo ALTI (exploding).

Nella fase di retropropagazione, il calcolo del gradiente per i pesi dei primi strati (es. w_1) tramite la **CHAIN-RULE** genera una lunga sequenza di moltiplicazioni dipendente dai pesi e dalle derivate delle funzioni di attivazione degli strati successivi. Ad esempio, in una rete a 4 strati:

$$\frac{\partial \mathcal{L}}{\partial w_1} = \text{ERR} \cdot (d\sigma)^4 \cdot w_4 \cdot w_3 \cdot w_2 \cdot x_1$$

↓ **PROBLEMA di INSTABILITÀ:**
Se analizziamo questo comportamento generalizzando a L strati, il gradiente contiene il termine:

$$\prod_{j=1}^{L} \sigma'(s_j) \cdot w_j$$

ESEMPIO (Vanishing Gradient con Sigmoide):
Se consideriamo l'uso della funzione **SIGMOIDE** $\sigma(x) = \frac{1}{1+e^{-x}}$, sappiamo che la sua derivata è $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

Il valore massimo assunto da questa derivata si ha in $x = 0$, ovvero $\sigma'(0) = \frac{1}{4}$. Pertanto:

$$|\sigma'(x)| \leq \frac{1}{4}$$

Supponendo che i pesi della rete abbiano modulo $|w_i| < 1$, il singolo termine della produttoria risulta limitato:

$$|\sigma'(x) \cdot w_i| < \frac{1}{4} \cdot 1$$

Ripetendo questo prodotto per tutti gli L strati della rete, otteniamo il limite:

$$\left| \prod_{j=1}^{L} \sigma'(x) \cdot w_j \right| < \left(\frac{1}{4} \right)^L = \frac{1}{4^L}$$

Essendo una frazione inferiore a 1, all'aumentare di L (reti molto profonde), la quantità $\frac{1}{4^L}$ tende rapidamente a zero. Di conseguenza, il gradiente per i primi pesi diventa talmente "piccolo" da svanire, rendendo impossibile un aggiornamento efficace dei parametri nei primi strati durante l'algoritmo di discesa del gradiente.

32) Spiega il funzionamento di una CNN, indicando il ruolo di ciascun tipo di layer (convoluzionale, pooling, lineare).

L'architettura standard di una **CNN (Convolutional Neural Network)** è progettata per gestire dati a "GRIGLIA" (es. immagini) ragionando direttamente a **VOLUMI (o TENSORI)**. La struttura prevede tipicamente la sequenza:

$$\text{INPUT } \underline{x} \longrightarrow \text{Conv} + \text{Pool} \longrightarrow \cdots \longrightarrow \text{Conv} + \text{Pool} \longrightarrow \text{MLP} \longrightarrow \text{OUTPUT } \underline{y}$$

Il ruolo dei singoli layer è il seguente:

- **LAYER CONVOLUZIONALE:**
  Elabora l'input multidimensionale applicando filtri locali. È definito dai seguenti **IPERPARAMETRI**:
  - **DEPTH** (d): profondità o numero di filtri.
  - **STRIDE** (s): passo di spostamento del filtro.
  - **ZERO-PADDING** (p): dimensione del contorno di zeri aggiunto attorno all'input.
  - **DIMENSIONE KERNEL**: $n \times n$ (con n dispari).
    Sfrutta il principio dei **PESI CONDIVISI**: lo stesso filtro $n \times n$ scorre su tutti i pixel mantenendo gli stessi pesi. Ad esempio, se $n = 3$, ci sono *SOLO 9 PARAMETRI* da addestrare per generare una determinata slice, indipendentemente dalla grandezza dell'immagine. L'output del blocco è dato da un'operazione lineare seguita dall'attivazione:

    $$c_{ij}^k = \sigma\left(W_k^{(1)} * \underline{x} + \underline{b}^{(1)} \right)$$

- **LAYER di POOLING:**
  Costituisce un filtro **NON-LINEARE** utilizzato per ridurre la dimensionalità spaziale. Il metodo standard è il **MAX-POOLING** (governato dai parametri n e s), che fa scorrere una finestra estraendo esclusivamente il valore massimo all'interno dell'intorno:

  $$x_{ij} \longmapsto \max(x_{i+m,j+m}) = p_{ij}^k$$

  (Nota: Durante l'algoritmo di backpropagation, poiché la funzione max non è strettamente differenziabile, i valori di input non massimi annullano il gradiente $\nabla \mathcal{L}$, mentre la derivata si propaga solo per il valore massimo estratto).
- **LAYER LINEARE (MLP):**
  Posto tipicamente alla fine della sequenza di estrazione delle *feature*, riceve in ingresso i dati elaborati dai blocchi convoluzionali e di pooling. Si comporta come una rete tradizionale (Fully Connected), combinando le informazioni estratte per calcolare la previsione e produrre l'output finale $\underline{y}$.